

Flutter と Dart プログラミング

Flutter は、美しいユーザーインターフェース (UI) を持つアプリを、Android と iOS の両方で動作するように簡単に開発できるフレームワーク。

Flutter の裏側には Dart というプログラミング言語が使われている。

Dart はシンプルで高速、そして初心者にも学びやすいように設計されている。

Dart プログラミングの基本概念

1. Dart 変数

- 情報 (数字やテキスト) を保存するために使う。
- `var`, `int`, `double`, `String` などの型で宣言できる。

```
var name = 'Alice'; // A string variable
int age = 30;       // An integer variable
```

2. Dart の演算子

- 演算子は、変数や値に対して加算、減算、比較、代入などの操作を行うために使用される。
 - ・ より多くの演算子や使い方については、[こちらをご覧ください](#)。

```
int sum = 5 + 3; // Addition
bool isEqual = (5 == 5); // Comparison
```

3. Dart のキーワード

- キーワードとは、`if`, `for`, `class`, `void` など、Dart であらかじめ意味が定義された特別な単語のこと。
- キーワードは変数名などには使用できません。
- より多くのキーワードについては、[こちらをご覧ください](#)。

4. Dart 内蔵タイプ

- Dart にはさまざまなデータを扱うための組み込み型が用意されている：

◇ 数値 ([Numbers](#)) : `int`, `double`

`int`: 整数 (小数なし) & `double`: 浮動小数点数 (小数あり)

```
int x = 10;
double y = 3.14;
```

◇ 文字列 ([Strings](#)) : `String`

テキストを表すために使用する。

```
String greeting = 'Hello, World!';
```

- ◇ 真偽値 ([Booleans](#)) : bool
true または false の値を持つ.

```
bool isActive = true;
```

- ◇ レコード ([Records](#)) : (値 1, 値 2)
複数の値を 1 つにまとめる簡単な方法.

```
var record = (10, 'hello');
```

- ◇ 関数 ([Functions](#)) : Function
関数は処理を行い, 結果を返すことができる.

```
int add(int a, int b) {  
    return a + b;  
}
```

- ◇ リスト ([Lists](#)/ 配列) : List
複数の値を順序付きで保存する.

```
List<String> colors = ['red', 'blue', 'green'];
```

join() メソッドを使うと, リスト内の要素を 1 つの文字列に結合できる.

```
List<String> fruits = ['Apple', 'Banana', 'Cherry'];  
print(fruits.join(', ')); // Output: "Apple, Banana,  
Cherry"
```

- ◇ セット ([Sets](#)) : Set
重複のない値の集まる.

```
Set<int> uniqueNumbers = {1, 2, 3};
```

- ◇ マップ ([Maps](#)) : Map
キーと値のペアを保存する.

```
Map<String, int> scores = {'Alice': 90, 'Bob': 85};
```

5. Dart Generic

- ジェネリクスを使うことで, 異なるデータ型に対応できる柔軟なコードが書ける.
- 例えば, 整数だけを保持するリストを定義するには:

```
List<int> numbers = [1, 2, 3];
```

6. Dart のエラー処理

- プログラム実行中に問題が起きたときに,それを管理・処理する方法.
- Dartでは try,catch,finally を使う.

```
try {
  int result = 10 ~/ 0;
} catch (e) {
  print('Cannot divide by zero: $e');
}
```

7. Dart OOP

- Dart は OOP に対応しており,クラスを使ってオブジェクトを定義できる.
- オブジェクトはプロパティ (変数) とメソッド (関数) を持つ.

```
class Car {
  String make;
  String model;

  Car(this.make, this.model);

  void displayInfo() {
    print('Car: $make $model');
  }
}

void main() {
  var myCar = Car('Toyota', 'Corolla');
  myCar.displayInfo();
}
```

列挙型 (Enum)

- Enum (列挙型) は,特定の固定された値を表す特別な型.
- 特定の値しか許可したくないときに便利.

```
enum CarType { sedan, suv, truck }

void main() {
  CarType myCarType = CarType.sedan;

  if (myCarType == CarType.sedan) {
    print('Your car is a sedan.');

```

8. Dart Null Safety

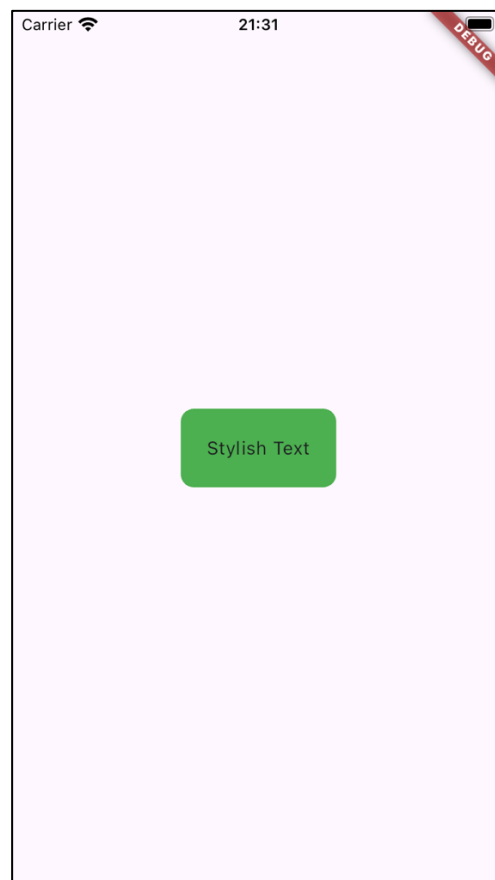
- 変数が「null」になることによるエラーを防ぐ仕組み.
- Dartでは変数が null を取れるかどうかを明示する必要がある.

```
String? nullableString; // This can be null
```

```
String nonNullableString = 'Hello'; // This cannot be  
null
```

- 例として Text ウィジェットが Container ウィジェットの中にネストされる：

```
Container(  
  padding: EdgeInsets.all(20),  
  decoration: BoxDecoration(  
    color: Colors.green,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Text('Stylish Text'),  
);
```



なぜ Flutter はウィジェット内部で関数やプロパティを使うのか？

- a. なぜウィジェットで関数やプロパティが使われるのか？
 - Flutter では、ウィジェット内部で関数やプロパティを渡すことができる。
 - UI（見た目）とロジック（動作）を1つの場所で定義できるので便利。
 - 従来の UI フレームワークのように、UI とロジックを分ける必要がない。
- b. 例：onPressed でアクションを追加する（ボタンの例）
 - ボタンウィジェットの onPressed に関数を渡すことで、クリック時の動作を指定できる。

- 見た目（UI）と動作（ロジック）を一箇所にまとめることができる。
- 以下はコード例です：

```
ElevatedButton(
  onPressed: () {
    print('Button clicked!'); // This function runs when
the button is pressed
  },
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 説明：
 - ◇ onPressed は関数を渡すためのプロパティ。
 - ◇ ボタンがクリックされると関数が実行される。
 - ◇ UI（テキスト）とロジック（クリック時の動作）が一緒に書かれている。
- c. コードを整理しやすくするために関数を分離する
 - 関数をウィジェットの外に定義し、参照することで、コードが整理され、保守性が向上する。
 - 関数が複雑な場合や、複数の場所で再利用する場合に便利。
 - 例えば、以下は関数を分けて使うコード例です：

```
void buttonClicked() {
  print('Button clicked!'); // This function runs when the
button is pressed
}

ElevatedButton(
  onPressed: buttonClicked, // Just reference the function
by its name
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 関数を分離するメリット：
 - ◇ 再利用性 → 複数のボタンやイベントで関数を再利用できる。
 - ◇ 明確さ → UI コードがスッキリし、デザインに集中できる。
 - ◇ 保守性 → ボタンの動作を変更する場合、関数を 1 箇所変更するだけで済む。
- d. 例：プロパティを使ったスタイリング（Text ウィジェットの TextStyle）
 - プロパティを使うことで、ウィジェットの見た目を簡単にカスタマイズできる。
 - デザイン用のファイルを別に作らなくても、TextStyle などのプロパティでスタイルを定義できる。
 - コードが読みやすく、変更しやすくなる。

- 以下はコード例です：

```
Text(  
  'Hello, Flutter!',  
  style: TextStyle( // TextStyle customizes the appearance  
of the text  
    fontSize: 24,    // Makes the text larger  
    fontWeight: FontWeight.bold, // Makes the text bold  
    color: Colors.blue, // Changes the text color to blue  
  ),  
);
```

- 説明:
 - ◇ style は TextStyle オブジェクトを受け取るプロパティ.
 - ◇ fontSize,fontWeight,color はテキストの見た目を定義するキー.
 - ◇ 別のスタイルシートを作らなくても,直接ウィジェット内でカスタマイズできる.

まとめ

- Flutter のキーと値のペアにより,カスタマイズと柔軟な設定が可能になる.
- デフォルト値を上書きすることで,UI の外観や動作を変更できる.
- Flutter の名前付きパラメータは,コードの可読性と保守性を向上させる.
- Flutter のキーと値の構造は JSON に似ているが,目的が異なる.
- ウィジェットのネストによって,UI 要素の構造を明確にできる.
- 関連するコンポーネントをひとまとめにすることで,コードが読みやすくなる.
- スタイル用ウィジェットでラップすることで,カスタマイズが容易になる.
- Flutter の UI はすべてネストされたウィジェットツリーで構築されている.
- ウィジェット内の関数は動作を定義する (例: onPressed) .
- 関数を分離することで,再利用性・明瞭性・保守性が向上する.
- ウィジェット内のプロパティは外観をカスタマイズする (例: Text の style) .
- Flutter は UI とロジックを一箇所に統合しているため,コードの管理がしやすい.

- メソッドのシグネチャが一致しない場合のミスを防ぐ.
- Java では明示的なメソッドシグネチャが必要だが,Dart はより柔軟.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- 説明:
 - ◇ @Override → Child の build() が Parent の build() を正しくオーバーライドしていることを保証する.
 - ◇ メソッド名が間違っていた場合,Java はエラーを出す.
- 主な違い:

機能	Java	Flutter
@override の用途	OOP メソッドのオーバーライド	Flutter ウィジェットのオーバーライド
エラー防止	誤ったメソッドシグネチャを防ぐ	タイプミスの検出を助ける
柔軟性	厳格なメソッド定義	より柔軟だが,@override で明確

ステップ6：build() メソッドとBuildContext の探索

a. build()メソッドとは？

- build() メソッドは,ウィジェットが画面にどのように表示されるかを定義する.
- “ウィジェットツリー” (UI を構成するウィジェットの階層構造) を返す.
- UI が更新される必要があるときに,build() メソッドが再び呼び出される.

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```



```
}
```

- 説明:
 - ◇ build() は MaterialApp ウィジェットを返し,アプリの UI 全体を定義する.
 - ◇ MaterialApp の中に Scaffold,Center,Text などのウィジェットが含まれる.
 - ◇ この構造がウィジェットツリーを形成し,UI のレイアウトを決める.
- b. BuildContext とは?
 - BuildContext は,ウィジェットがウィジェットツリーのどこにあるかを示す情報を提供する.
 - 継承されたウィジェットにアクセスしたり,ナビゲーションを管理したりできる.
 - build() メソッドの必須パラメータである.
- c. build()が UI を更新し続ける仕組み
 - Flutter は UI の変化を検出すると build() を再び呼び出す.
 - UI の最新状態を確実に表示するために build() を利用する.
 - Flutter は宣言的 UI モデルを採用しており,手動で UI を変更するのではなく,再構築する.
- d. 比較: Java における UI の構築方法との違い
 - Java (Android) では,通常,UI の更新は View オブジェクトの変更によって行われる.
 - Flutter では,UI は宣言的に構築され,build() を使って再構築される.
 - Flutter のアプローチにより,UI の更新が構造化され,予測しやすくなる.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- 主な違い

機能	Java (Android)	Flutter (Dart)
UI の構造	XML や Java コードで View を作成	build() メソッドでウィジェットツリーを返す
状態管理	View を手動で更新	状態が変わると UI が自動更新
柔軟性	View 階層を変更する必要がある	build() によりシンプルに管理

ステップ 7: Flutter における return の役割

- a. build()における return の役割とは?
 - build() メソッド内で return を使って,ウィジェットが表示する内容を指定する.